

CPT202 Assignment 2

Group Report for Software Engineering Group Project

Project B: Music Resource Management

Melodify - A secure, personalized and user-friendly system

4/26/2025

<http://101.37.37.8:8080>

Group 7

Submitted by: Xuexin Zhan, 2251792

Group Members:

1. Chenrui Ding, 2252734
2. Jin Zhou, 2251739
3. Xinyi Yang, 2251657
4. Xu Chen, 2257453
5. Xuexin Zhan, 2251792
6. Yiyi Zhu, 2252441
7. Youlong Xu, 2252978
8. Zichen Qiu, 2252705

2024/2025 Semester 2

Contents

1	Introduction	1
	Introduction	1
1.1	Background and Requirements	1
1.2	Aims of The Project	1
1.3	User Characteristics	2
1.4	Assumptions and Dependencies	2
1.5	Project Scope	2
1.6	Project Risks	3
2	Architectural Design	3
	Architectural Design	3
2.1	System Architecture	3
2.2	Frontend System	4
2.3	Backend System	4
2.4	High-level Database Design	5
2.4.1	Detailed Relationship Analysis	5
2.4.2	Design Principles	5
3	Software Design	6
	Software Design	6
3.1	Software Modules	6
3.2	High-level Process Flow	6
3.3	Software Support Services	7
3.4	Coding Structure and Convention	8
3.5	Software Configuration and Production Environment	8
4	Software Testing	9
	Software Testing	9
4.1	Unit Test	9
4.2	Integration Testing	10
4.3	Acceptance Testing	10
5	Conclusion and Future Work	11
	Conclusion and Future Work	11
	Appendix A: Admin Account and Login Helps	12
	Appendix B : UML Diagram	13
	Appendix C: Screen Shots of Music Management System	13
	Appendix D: Configuration Files	18
	Appendix E: Testing Code and Result	20
	Appendix F: Sprint Backlogs	22
	Appendix G: Secure encryption	23
	Appendix H: Member Contributions Form	24

1 Introduction

1.1 Background and Requirements

Driven by the wave of digitalization, the field of digital music has developed rapidly, and the way of acquiring, storing and sharing music resources has undergone tremendous changes [1]. With the growing demand of users for personalized and efficient music resource management, the development of music resource management systems has become increasingly important.

However, there are still many problems with the current music resource management system. In terms of user experience, the user authentication process of some platforms is cumbersome, which affects the efficiency of use; the music upload and organization functions are not smart enough, which makes it difficult for users to quickly organize the huge music library [2]. At the system management level, administrators lack efficient tools for content review and management, and user privacy and data security also face potential risks, such as data leakage and illegal access [2]. The existing solutions mainly include optimizing the user interface design, simplifying the authentication process, improving the accuracy of music classification and search; strengthening data encryption technology to ensure user information security [3]. Nevertheless, these methods lack systematic integration and are difficult to fundamentally meet the diverse needs of users and administrators.

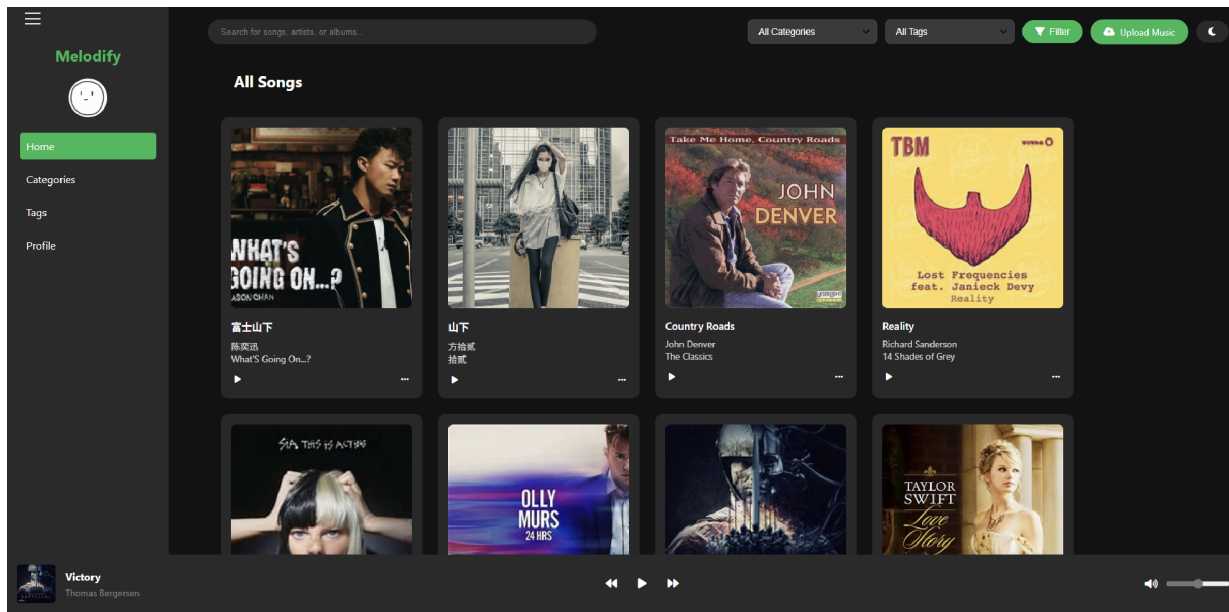


Figure 1.1: The User Interaction Interface of the Music Management System (Melodify)

In this project, we are committed to creating a comprehensive and user-friendly music resource management system (Melodify). In terms of user authentication, it ensures safety and convenience; the music upload and organization functions will support tag generation and classification, which is convenient for users to quickly organize; in terms of system management, it provides administrators with a visual content review and user management panel to achieve efficient management.

Our contributions:

- **We construct a comprehensive, high-performing, and secure integrated music management solution, effectively propelling music operators to achieve efficient management and accelerate the process of digital transformation.**
- **Balance between user experience and management needs**

1.2 Aims of The Project

The primary aim is to develop a comprehensive music resource management system that provides an intuitive and engaging experience for end-users while offering robust administrative capabilities for system administrators. The project seeks to achieve the following specific objectives:

1. **User-Centric Design:** Create a user-friendly interface that allows end-users to easily navigate the system, register, log in, and manage their music collections effortlessly.
2. **Efficient Music Management:** Implement features that enable users to upload, categorize, and tag their music files, ensuring that they can organize their resources effectively.

3. **Enhanced Search Functionality:** Develop advanced search and filtering options that allow users to quickly locate specific music tracks based on various criteria.
4. **Data Security and Privacy:** Ensure that user data and uploaded content are securely protected through robust security measures, including encryption and secure authentication.
5. **Administrative Oversight:** Provide system administrators with tools to monitor user activity, review uploaded content, and enforce compliance with platform rules.
6. **User Feedback Mechanism:** Establish a feedback system that allows users to share their comments and suggestions, facilitating continuous improvement of the platform.

1.3 User Characteristics

The needs for end-users and system administrators are shown in Figure 1.2.

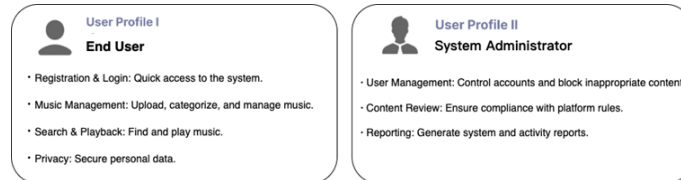


Figure 1.2: User Profile

End Users seek an intuitive and seamless experience. They want to easily register, log in, and upload their music files. The system should allow users to organize their resources with relevant metadata (like title, artist, and album), and provide them with options to categorize and tag their music. Additionally, users need efficient search and filtering capabilities to find specific music quickly. What's more, it is essential to ensure that their personal information and uploaded content are protected. Finally, users should be able to play their music directly from the system.

System Administrators are responsible for managing user accounts and content. They need tools to review and moderate uploaded music, ensuring compliance with platform rules. Administrators also have the authority to lock or unlock user accounts based on content violations. In addition, to generate insights into user activities and system performance, administrators require data reporting capabilities.

1.4 Assumptions and Dependencies

It is assumed that users are familiar with basic digital device operations and can independently complete system registration, login, music upload, and other operations. Meanwhile, it is presumed that users will search for and play music according to their personal preferences and use classification tags to manage their music libraries. Regarding administrator behavior, it is assumed that administrators can proficiently use management tools to regularly review user accounts and music content and handle violations. Moreover, it is presumed that administrators can reasonably manage user permissions and platform data based on system rules and user feedback. We assume that the server has sufficient performance to support a large number of users' concurrent access and music data storage and transmission, and can run stably on mainstream operating systems and browsers.

This project relies on Alibaba Cloud services to acquire cloud computing resources that can be automatically scaled according to traffic, thereby enabling it to adapt to fluctuations in user traffic. In addition, it depends on Docker and Docker Compose for containerized management, which ensures the stability and portability of the environment. Moreover, to achieve team collaboration, it relies on Git and GitHub for code management and version control. Regarding data processing and persistence, it depends on Spring Boot and MySQL, and with the assistance of JPA and MyBatis-Plus, it can handle data effectively and ensure its persistence. For the purpose of safeguarding user authentication and data privacy security, it relies on Spring Security and Spring Boot Starter Mail. Finally, to enable seamless audio playback of MP3 files, it depends on JLayer.

1.5 Project Scope

The functional scope of our system includes profile management, music resource management, and user feedback functions. Figure 1.3 shows the overall functions:

- Users are first required to register an account via email. After registering and logging in, users can enjoy the subsequent functions, including categorizing and uploading music, searching for music, playing music, and launching user feedback service, etc. Meanwhile, as a user, you can also manage your personal data.
- For administrators, the system provides an easy-to-use backend management system. They are able to view all the music resources uploaded by users in order to review the content for compliance. If uploaded music resources are illegal, administrators can block content to ensure the security of the system.

- In terms of user feedback, the system can collect users' comments and suggestions on this system and make timely response and improvement, which helps to improve the quality of the system and enhance user experience.

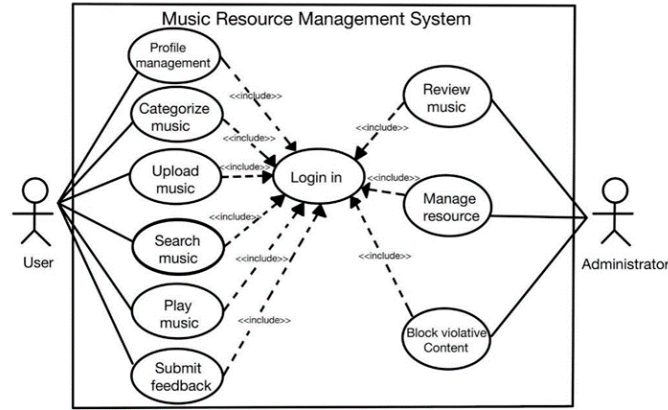


Figure 1.3: Use Case Diagram

This project clearly excludes the following aspects:

- This project does not involve the acquisition and management of music copyrights. It only provides a music playback platform and does not engage in any music copyright - related business.
- This project is not responsible for the procurement and maintenance of hardware devices. The system will operate entirely on cloud servers and does not cover the management of local hardware devices.
- This project focuses on the development of a web - based online music playback system and does not include the development of native mobile applications (such as iOS and Android applications).

1.6 Project Risks

In the process of completing this project, we mainly used Agile development methodology to improve the development efficiency, but at the same time there are also risks in our project. In other words, user requirements change and developers continue to improve, the project becomes more functional and broader in scope, and these changes may bring some risks and vulnerabilities.

To cope with these risks, we use Session Management, Authentication to improve the security of the system. At the same time, we use unit testing, integration testing, functional testing and security testing to find potential loopholes, and continuously improve the improvement, so as to reduce the risk, improve security, and ensure the smooth progress of the project and final delivery.

2 Architectural Design

2.1 System Architecture

This music management web application employs a three-tier architecture, encompassing Gateways & Proxies, App Side, and Storage & Cache, each contributing to the system' s security, performance, and scalability.

In the Gateways & Proxies layer, UFW establishes basic access control to fortify network security by filtering unauthorized traffic. Ali Safety enhances this by offering advanced inter - domain security control and traffic cleansing, warding off malicious attacks. Nginx functions as a reverse proxy, adeptly managing incoming requests through dynamic routing and request forwarding. It also excels in abnormal interception, blocking malicious requests to maintain system stability while optimizing network traffic via traffic cleaning.

Moving to the App Side layer, the MVC framework facilitates a clear separation of data processing (Model), user interface (View), and request handling (Controller), boosting code maintainability. It supports diverse user and admin functions, such as music playback, uploads, tag searches, user account management, and administrative audits. Integrated as a distributed messaging system, Kafka ensures seamless data flow processing. For instance, it triggers real-time notifications for music updates or uploads, enabling smooth interactions across modules.

In the Storage & Cache layer, the MySQL database serves as the backbone for data storage, securely managing music files, user information, and audit logs while supporting efficient query and management operations. Redis complements this by acting as a high - performance cache and message broker. It accelerates access to frequently used data like user session details and popular music tags, while also enabling high - concurrent processing to enhance overall system responsiveness.

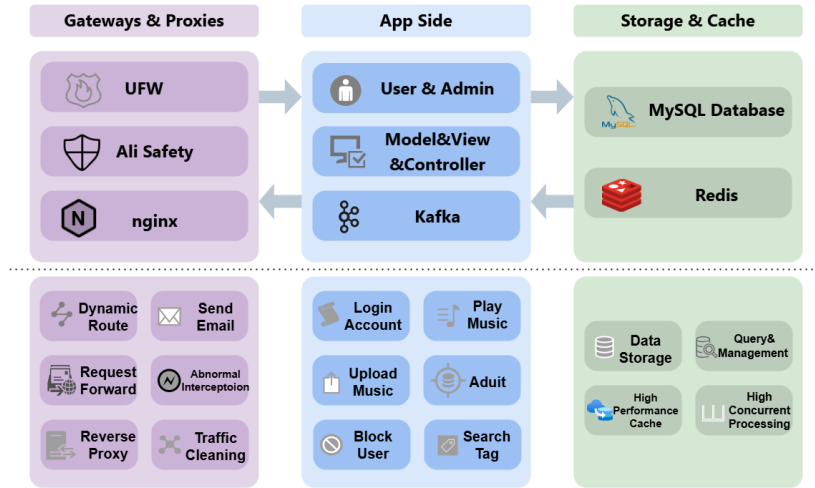


Figure 2.1: System Architecture Diagram

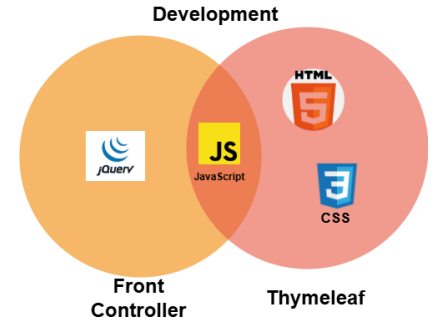


Figure 2.2: Frontend Architecture

2.2 Frontend System

The frontend system is divided into the Front Controller and View, which are developed using a combination of technologies as shown in the diagram.

In the Front Controller, JavaScript processes user input, determining whether to interact with the backend or handle tasks directly in the frontend. For instance, when a user in the admin interface attempts to manage music playlists or a regular user tries to play music, JavaScript assesses the need for backend interaction. jQuery is also integrated to simplify DOM manipulation and event handling. It aids in efficiently capturing user actions, such as clicks or form submissions, in both admin and user interfaces. For example, in the user interface, jQuery can smoothly handle the dynamic loading of music lists.

Regarding the View, we leverage Thymeleaf along with HTML and CSS. Thymeleaf allows us to create dynamic views by integrating server-side data into HTML templates. For the admin interface, Thymeleaf helps render pages for managing music resources, such as uploading new tracks or editing metadata. In the user interface, it displays music libraries, playlists, and user-specific settings. CSS, combined with HTML, is used to style these views. We can define consistent styles for buttons, menus, and music display components across both admin and user interfaces. For example, in the user interface, CSS ensures that music playback controls (play, pause, skip) have a uniform and intuitive appearance. Moreover, responsive design principles are applied using CSS, so whether admins manage music on a desktop or users access the app on a mobile device, the layout adjusts appropriately. This ensures a seamless experience in both admin operations (like checking music statistics) and user interactions (such as browsing music catalogs).

2.3 Backend System

The backend system (Figure 2.3) consists of four layers: **Controller**, **Service**, **Repository**, and **Models**. When a frontend client sends an HTTP request, it first reaches the **Controller Layer**, which validates and routes the request. The **Controller** then hands off the processing to the **Service Layer**, where the core business logic, including data transformations and validations, takes place. For database operations, **services** interact with the **Repository Layer**, which abstracts data persistence tasks and translates object-oriented calls into database queries. The **repositories** work with **Models** to ensure clean data mapping. Responses flow back through the layers—**repositories** deliver data to **services**, which process results and send formatted responses back to the client. This structure promotes separation of concerns, maintainability, and scalability while ensuring integration through well-defined interfaces.

Presently, it is imperative to conduct an in - depth analysis of the intricate details inherent in each technical module, thereby facilitating a comprehensive and profound comprehension of the backend system's architecture and operational mechanisms:

- For **File Uploads**, it employs Spring's **MultipartFile** interface, simplifying file data processing. This interface automates multipart data parsing, extracting metadata like file names and content types, and enabling effective file transfers to server storage. It is used for **user avatar uploads** and **music submissions**, ensuring consistent binary data handling and supporting multiple file formats.
- For **Email Operations**, the system integrates the **JavaMail API**, enabling reliable messaging for workflows like **account activation**, **password resets**, and **system notifications**. It supports plaintext and HTML templates with configurable SMTP settings for various providers. The service includes retry mechanisms for failed deliveries, queue management for high volumes, and ensures message delivery during peak loads [4].

- For **Authentication and security**, the system uses **JWT (JSON Web Tokens)** for user sessions and verification [5]. It supports various flows like password resets and email verification, forming the backbone of the system's security model and protecting user data while allowing seamless access to authorized features.
- For **Data Access**, the system employs **Spring Data JPA** to streamline database interactions. This powerful **object-relational mapping (ORM) framework** significantly reduces boilerplate code by providing out-of-the-box implementations for common CRUD operations [6]. Its entity manager manages database connections and transactions, ensuring resource cleanup and data integrity.

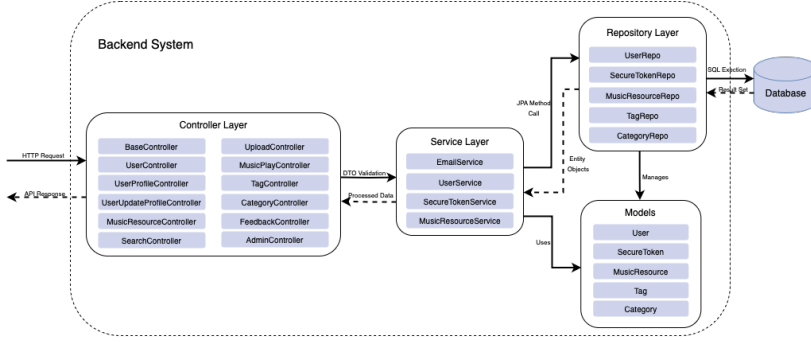


Figure 2.3: Backend Architecture

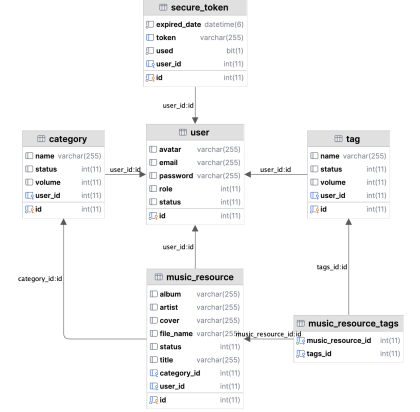


Figure 2.4: ER Diagram

2.4 High-level Database Design

The database (Figure 2.4) consists of six interconnected tables that form the foundation of a secure music management system:

1. **user**: Stores user account information.
2. **secure_token**: Manages security tokens for authentication workflows.
3. **category**: Organizes music resources into user-defined groups, such as pop, folk and vocal.
4. **tag**: Provides flexible labeling system for music resources, such as cheerful and heartbroken.
5. **music_resource**: Contains core music track information.
6. **music_resource_tag**: Junction table for many-to-many tag relationships between tag and music track.

2.4.1 Detailed Relationship Analysis

1. **User → Categories**: A **one-to-many** relationship enforced by the **user_id** foreign key in the **category** table, allowing each user to create and manage multiple music categories while ensuring each category is exclusively tied to a single user.
2. **User → Music Resources**: A **one-to-many** relationship exists via the **user_id** foreign key in the **music_resource** table, ensuring that each track belongs to one user while allowing multiple uploads per user for clear ownership.
3. **User → Tags**: A **one-to-many** relationship is established through the **user_id** foreign key in the **tag** table, preventing naming collisions and allowing personalized tagging systems.
4. **User → Secure Token**: A **one-to-many** relationship via **user_id** gives each user multiple authentication tokens, enhancing security while keeping usability intact.
5. **Category → Music Resources**: A **one-to-many** relationship enforced by the **category_id** foreign key in the **music_resource** table, allowing multiple tracks in a category, and only one category for each track.
6. **Music Resources ↔ Tags**: A **many-to-many** relationship is established through the **music_resource_tags** junction table, enabling tracks to have multiple tags and tags to be reused for a flexible classification system.

2.4.2 Design Principles

Our database design strictly adheres to the third normal form (3NF) [7]. Separate tables are created for users, security tokens, categories, tags, and music resources respectively, and a junction table (**music_resource_tags**) is used to handle the many-to-many relationship between tracks and tags. This structure not only ensures data integrity but also minimizes data duplication to the greatest extent. In addition, referential integrity is strictly maintained through the setting of foreign keys. These foreign keys enforce valid relationships between tables and prevent the

occurrence of orphaned records, thus guaranteeing the consistency of the database. To enhance extensibility, status fields (supporting workflows such as active/inactive states), volume fields (used for performance indicator statistics), and a flexible tagging system that supports dynamic categorization are introduced in the design. Finally, the user-centric concept permeates the entire design, and the `user_id` field is uniformly included in all tables. This design ensures the secure isolation of data in a multi-user environment and improves the security and usability of the system.

3 Software Design

3.1 Software Modules

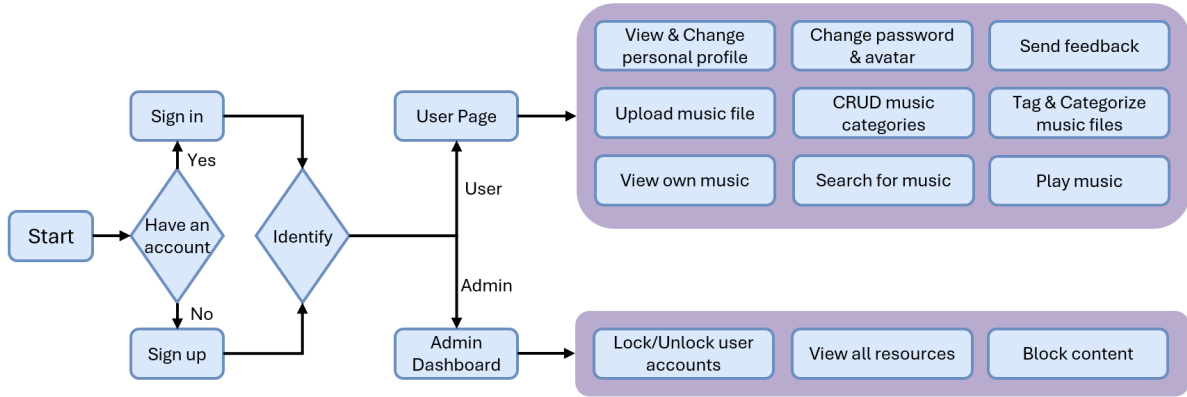


Figure 3.1: Software modularization

The Music Resource Management system employs a role-driven modular architecture, as illustrated in Figure 3.1, to balance security, usability, and scalability. The application is divided into User and Admin modules, supported by shared backend services. Specifically, a centralized authentication module is responsible for handling secure registration, email verification, and login processes. Meanwhile, a resource management service coordinates file uploads and metadata indexing. To ensure security, the system incorporates Role - Based Access Control (RBAC), input sanitization, and encryption techniques.

These architectural elements work in tandem to enable distinct user experiences tailored to different roles. In details, regular users access features such as profile updates, music uploads (MP3 files with metadata), tagging, categorization, and search/playback tools, all within a controlled environment. Admins, conversely, utilize a dedicated dashboard to review flagged content, lock accounts for violations, and manage global resources. Role-based access control (RBAC) is enforced at both UI and API levels to prevent unauthorized interactions.

An alternative function-centric design—grouping features like “upload” or “search” as standalone modules—was considered but rejected. This approach risked exposing admin tools to users and complicating access control. The current role-based modularization isolates workflows, simplifies maintenance, and enhances scalability. For instance, adding new roles (e.g., “Auditor”) or features requires minimal architectural changes.

Therefore, this design supports evolving requirements while maintaining robust data isolation and a streamlined user experience.

3.2 High-level Process Flow

The software architecture operates through two distinct workflows for users and administrators, as detailed in Figure 3.2 and Figure 3.3. You can refer to Appendix A to view the Admin Account and Login Helps and refer to Appendix B to view the Sequence Diagram For Music Resource Management.

For user interactions, the system prioritizes secure access and account initialization. Registered users undergo multi-layered authentication with sub-second response times to access core functionalities, while unregistered individuals initiate access through a structured registration interface. This process enforces automated validation of password complexity and email uniqueness, with new accounts remaining inactive until users complete email verification via an activation link. Full system privileges are granted only after successful confirmation of the submitted credentials.

Administrative operations focus on hierarchical content governance through a dedicated management console. Following user-submitted music uploads (including metadata such as album identifiers and cover art specifications), administrators monitor real-time updates to user accounts and digital assets. Their privileges extend to modifying account states (e.g., suspending or reactivating profiles) and moderating content (e.g., flagging or approving resources), with every operational change permanently logged in the transactional database.

The architecture enforces strict role-based segregation through three design principles: distinct interfaces for each user class, isolated code execution environments, and modular fault containment mechanisms. This approach

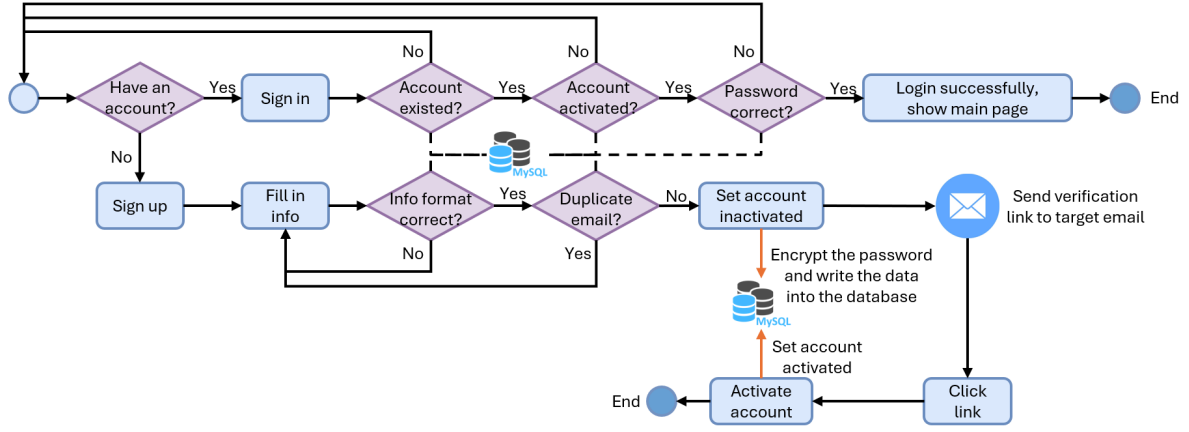


Figure 3.2: User sign in/up process

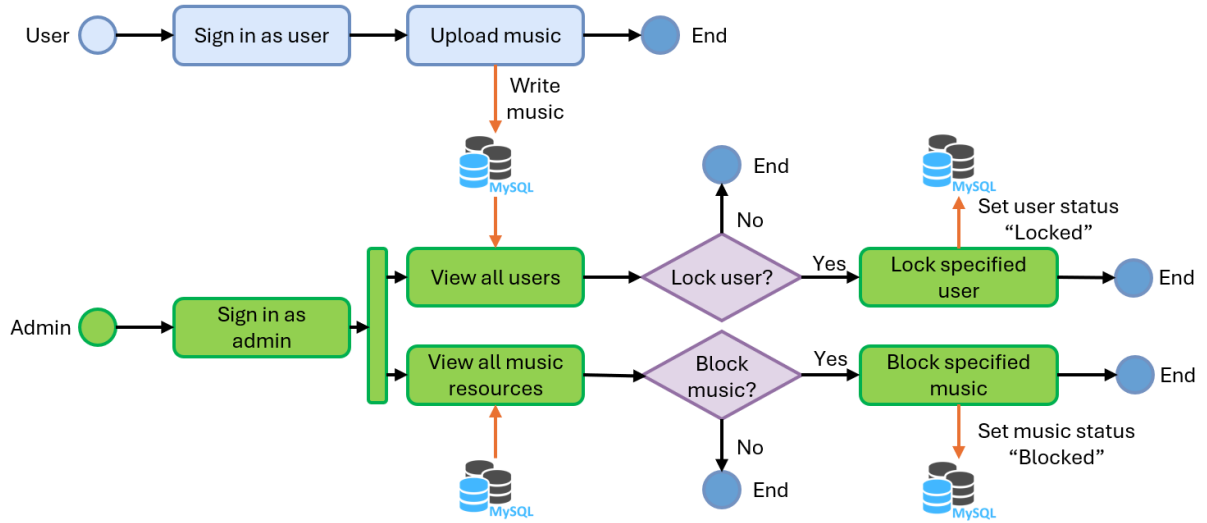


Figure 3.3: Admin Lock user/Block music process

ensures both system integrity and consistent user experience by preventing operational interference between roles while maintaining specialized workflows. All components are optimized for reliability, with audit trails and validation protocols embedded at critical interaction points.

3.3 Software Support Services

To make our music resource management system more efficient and stable, we use the following services:

- **CI/CD Pipeline:** The CI/CD pipeline automates the entire process of taking code from commit to build to test to deployment. (As shown in Figure 3.4) Developers write code in their local environment and then push this to GitHub, reducing the length of the traditional development model. The automated CI/CD process reduces human error and ensures the reliability of the development process through the mechanisms of continuous integration and continuous deployment.
- **Nginx reverse proxy:** Nginx acts as a reverse proxy server and forwards the request to the back-end according to the rules after receiving the client's request; the back-end server receives the request and processes it accordingly; Nginx then returns the back-end server's response to the client. The client only communicates with Nginx and does not interact with the back-end server to enhance system security.
- **Docker:** Docker packages application code, dependency libraries, configuration files, etc. into images through container technology, which realizes the consistency of development, testing and production environments so that each part does not affect each other without worrying about the differences in environments. Developers can deploy complex service stacks with a single click through the standard environment, which greatly improves development efficiency while avoiding errors caused by version mismatch.

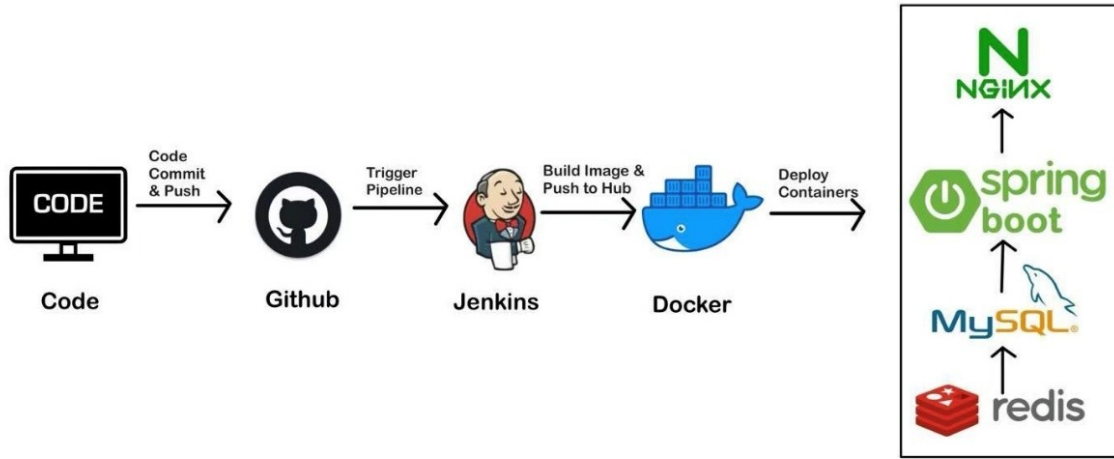


Figure 3.4: CI/CD pipeline

- **MySQL and Redis:** Read operations prioritize accessing the Redis cache and querying MySQL if there is no hit; write operations update MySQL first, then delete or update the Redis cache to avoid subsequent reading of old data. The ultimate consistency of data is achieved through an asynchronous synchronization mechanism. This collaborative model leverages the reliability of MySQL and the efficiency of Redis.

3.4 Coding Structure and Convention

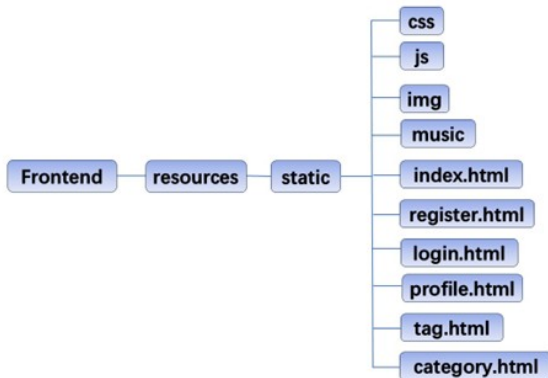


Figure 3.5: Front-end part

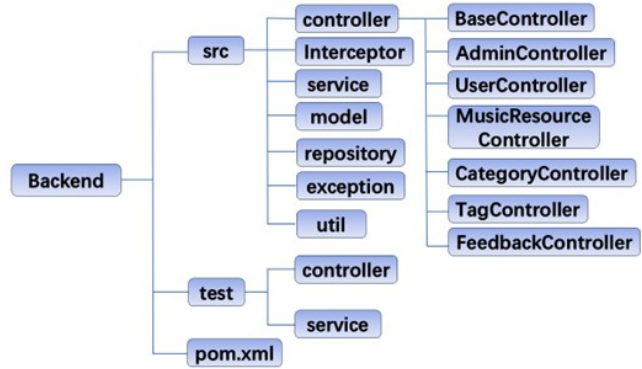


Figure 3.6: Back-end part

The project adopts layered architecture and modular development, the front-end builds the resource system, and the back-end follows SpringBoot architecture specification.

The front-end part is shown in Figure 3.5: all static resources are uniformly stored in resources/static/ directory, CSS using BEM naming, JavaScript modular camel naming and abstraction of tools, audio classified according to the scene; HTML entry mandatory semantic tags, and ultimately achieve the classification of resource storage, high performance and high quality development goals.

The back-end part is shown in Figure 3.6: the core structure consists of three parts: src, test and pom.xml. controller under src is the project's request scheduling center; test directory provides unit test coverage for the controller and service modules; pom.xml focuses on the management of the dependency library version with the construction of the process.

3.5 Software Configuration and Production Environment

The software is configured in accordance with CI/CD principles. CI is implemented by Jenkins, and CD is orchestrated via docker-compose.yml, including nginx, springboot, mysql and redis. The project is configured with a reverse proxy to redirect traffic from public network to the server intranet. Docker logs is used for centralized logging. See Appendix D: Configuration Files.

Table 1: Server Configuration

Field	Description
Operating System	Ubuntu 22.04
Core	4 Cores
RAM	8G
Bandwidth	5 Mbps
Max Upload Size	10MB
Max Request Size	20MB
JDK Version	JDK 17
Docker Version	26.1.3

Table 2: Software Configuration

Field	Version	IP	Port
Jenkins	2.492.3 LTS	127.0.0.1	8081
Nginx	1.27.5	172.30.0.4	80,443
Mysql	8.0.18	172.30.0.4	3306
Redis	6.2.12	172.30.0.5	6379
Springboot	3.4.3	172.30.0.6	8080

4 Software Testing

The Software Testing Life Cycle (STLC) is a crucial process in software development, consisting of several stages from test planning to evaluation. The six main stages in STLC are:

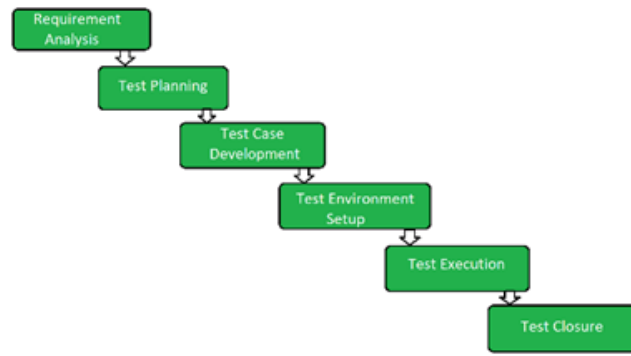


Figure 4.1: The Software Testing Life Cycle (STLC) [8].

1. **Requirement Analysis:** We collaborated with project stakeholders to review the functional requirements for modules, ensuring all necessary features are included.
2. **Test Planning:** We created a test plan to define the scope, objectives, and strategies for testing, focusing on the effectiveness of the modules and addressing potential issues, such as the security of user registration and login.
3. **Test Case Development:** We developed test cases based on the requirement analysis to validate the functionality of the modules, covering both positive and negative scenarios.
4. **Test Environment Setup:** We established a test environment that mirrors the production environment to ensure reliable test results.
5. **Test Execution:** We executed the test cases and recorded the results to verify if the software meets the requirements.
6. **Test Closure:** We evaluated the test results to confirm that all defects have been resolved and generated a test report summarizing lessons learned to improve future testing processes.

4.1 Unit Test

During the development of the Music Resource Management System, we conducted unit tests on key components such as user registration, login, music upload, and playback management using the JUnit framework. Each method was tested independently, including edge cases like null values, empty strings, and invalid inputs. We validated user authentication, music file uploads and downloads, and playback controls, ensuring proper functionality. Regression testing ensured that new features didn't break existing ones. Important test code has been included in the appendix. These tests helped ensure the system's correctness, completeness, and robustness, providing a reliable, high-quality experience for users. Please see Appendix E for details.

4.2 Integration Testing

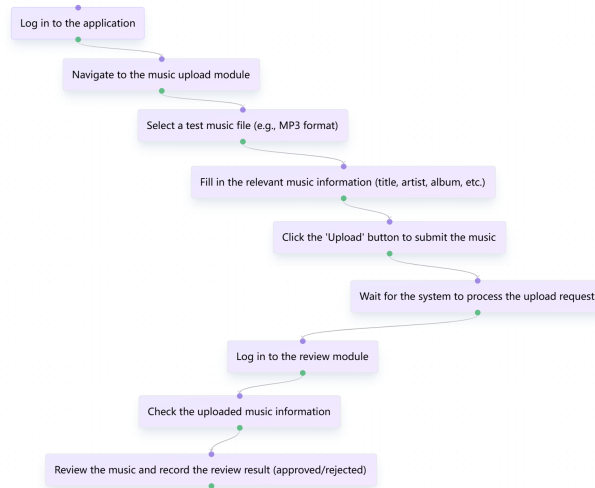


Figure 4.2: Test Steps

Purpose: Integration testing can verify the interactions between modules function correctly, ensuring the system works together to meet user needs.

Scope: It covers the integration of the music upload with review and classification modules, as well as the integration of the search and playback modules, focusing on the accuracy of data transmission and the timeliness of system responses.

Executed Modules: Music Classification, Music Upload, Music Playback, User Login and Registration, Music Search and Filtering, Profile Management, Admin Review and Management, User Feedback and Help.

Test case: Music upload and Music review.

Test Purpose: Verify that after a user uploads music, the review module can correctly receive and process the uploaded music information.

Test Steps: Shown in Figure 4.2.

Expected Results: The information of uploaded music should be properly displayed in the review module, including title, artist, album, etc. Admin should be able to review the uploaded music and return the corresponding review result.

Actual Results: The review module successfully received and processed the uploaded music information after the user uploaded the music. The uploaded music details, including title, artist, album, etc., were correctly displayed in the review module. The admin was able to access the uploaded music and complete the review, returning the corresponding review result.

4.3 Acceptance Testing

The acceptance testing phase of our Music Resource Management System involves ensuring the system meets user and stakeholder expectations. Key testing aspects include:

- **Functionality Testing:** We tested core functions like user registration, login, music uploads, playback management, and personalized recommendations using MockMvc. Each feature, including audio file uploads and playback controls, was validated to ensure expected behavior.
- **User Experience Validation:** We gathered feedback through surveys and usability testing to ensure the interface is intuitive.
- **Performance Evaluation:** Tests under normal and high-load conditions confirmed system stability and responsiveness with multiple users. Database optimization tests ensured efficient handling of large data volumes.
- **Security and Stability Checks:** Basic security testing was completed, with plans to improve protection against malicious file uploads, SQL injection, and XSS attacks.
- **Cross-Device Compatibility & Multi-Language Support:** We will extend testing to major browsers and devices, and introduce multi-language support (starting with English and Chinese) for a global user experience.

Therefore, these acceptance tests ensure that the system meets high standards of functionality, performance, and user experience. We will continue to enhance security and compatibility in future updates.

5 Conclusion and Future Work

In the development of our music management web application, we have successfully implemented a comprehensive set of features and security mechanisms to ensure both functionality and user privacy protection. By establishing secure registration and login protocols, complemented by email verification procedures, we have built a sturdy barrier against unauthorized access, effectively safeguarding user information. The application further enables users to conduct personalized music uploads and, with the aid of an advanced tag-based categorization system, not only streamlines the music organization process but also enhances music searchability and discoverability. Meanwhile, we have integrated resource moderation tools and user feedback channels, creating a dynamic and user-centric music resource management environment. These elements collectively form a reliable service framework, and its stable, error-free operation in real-world scenarios demonstrates the practical feasibility of our system.

However, like any complex system, our music management application has its limitations. The current digital integration strategies are relatively basic, which restricts the application's commercial expansion potential and makes it difficult to integrate with a wider range of digital platforms and services. Additionally, although the user interface can perform basic functions, its explainability is insufficient to fully meet the diverse needs of users from different professional and research backgrounds. This may lead to an unintuitive user experience and, especially in more specialized usage scenarios, may hinder the improvement of user satisfaction and the promotion of the application.

To address these limitations and drive future development, we have formulated a series of strategic improvement plans. In terms of enhancing commercial viability, we will explore advanced digital integration strategies, enabling seamless integration with e-commerce platforms, music streaming services, and other relevant digital channels. Regarding interface design, we will involve incorporating more intuitive visual cues, interactive tutorials, and clear guidance mechanisms, making the application more accessible and user-friendly to a broader user base. Through these planned improvements, we aim not only to overcome existing limitations but also to comprehensively enhance the overall performance and user experience of our music management web application.

References

- [1] W. Liu, "Integrating data mining with internet of things to build the music project management system," *Journal of Ambient Intelligence and Humanized Computing*, pp. 1–13, 2020.
- [2] E. Pizzolitto, "Music in business and management studies: a systematic literature review and research agenda," *Management Review Quarterly*, vol. 74, no. 3, pp. 1439–1472, 2024.
- [3] D. Afchar, A. Melchiorre, M. Schedl, R. Hennequin, E. Epure, and M. Moussallam, "Explainability in music recommender systems," *AI Magazine*, vol. 43, no. 2, pp. 190–208, 2022.
- [4] E. R. Harold, *JavaMail API: Sending and Receiving Email with Java*. " O'Reilly Media, Inc.", 2013.
- [5] S. Ahmed and Q. Mahmood, "An authentication based scheme for applications using json web token," in *2019 22nd international multitopic conference (INMIC)*. IEEE, 2019, pp. 1–6.
- [6] C. Tudose and C. Odubăşteanu, "Object-relational mapping using jpa, hibernate and spring data jpa," in *2021 23rd International Conference on Control Systems and Computer Science (CSCS)*. IEEE, 2021, pp. 424–431.
- [7] N. A. R. Amato, "Mastering database normalization: A comprehensive exploration of normal forms," *Researchgate*, 2023.
- [8] GeeksforGeeks Contributors, "Software testing life cycle (stlc)," Feb. 2024, accessed: 2025-04-23. [Online]. Available: <https://www.geeksforgeeks.org/software-testing-life-cycle-stlc/>

Appendix A: Admin Account and Login Helps

Admin Email : admin@123.com

Admin Password : !12345678

Regular User Email : Youlong.Xu22@student.xjtlu.edu.cn

Regular User Password : !12345678

For new user registration, your password needs to contain one uppercase letter, one lowercase letter, and one special symbol : ! # \$ % &

Appendix B : UML Diagram

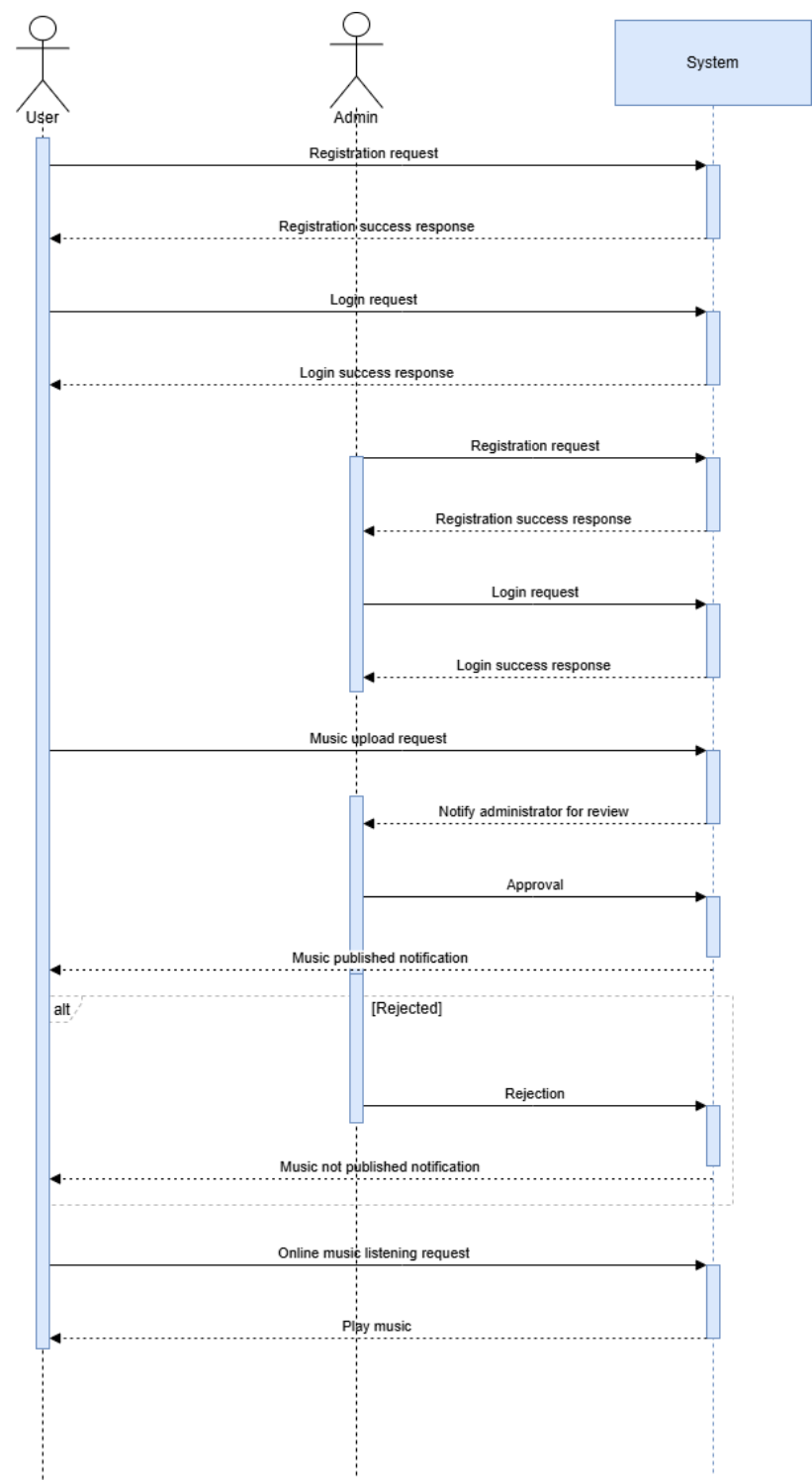


Figure Apd.1: Sequence Diagram For Music Resource Management

Appendix C: Screen Shots of Music Management System

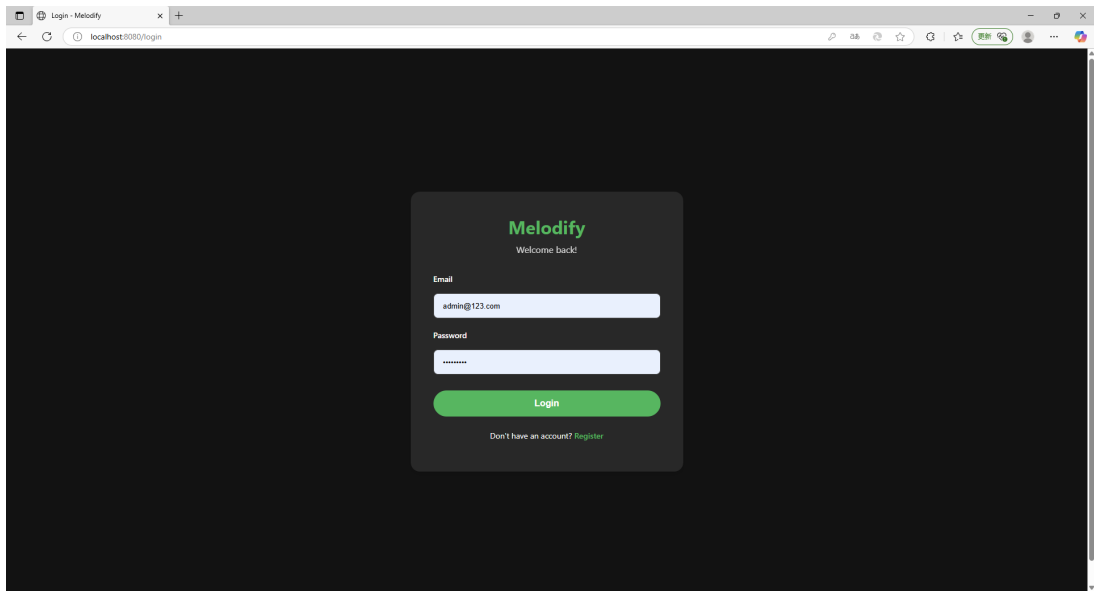


Figure Apd.2: Login Page

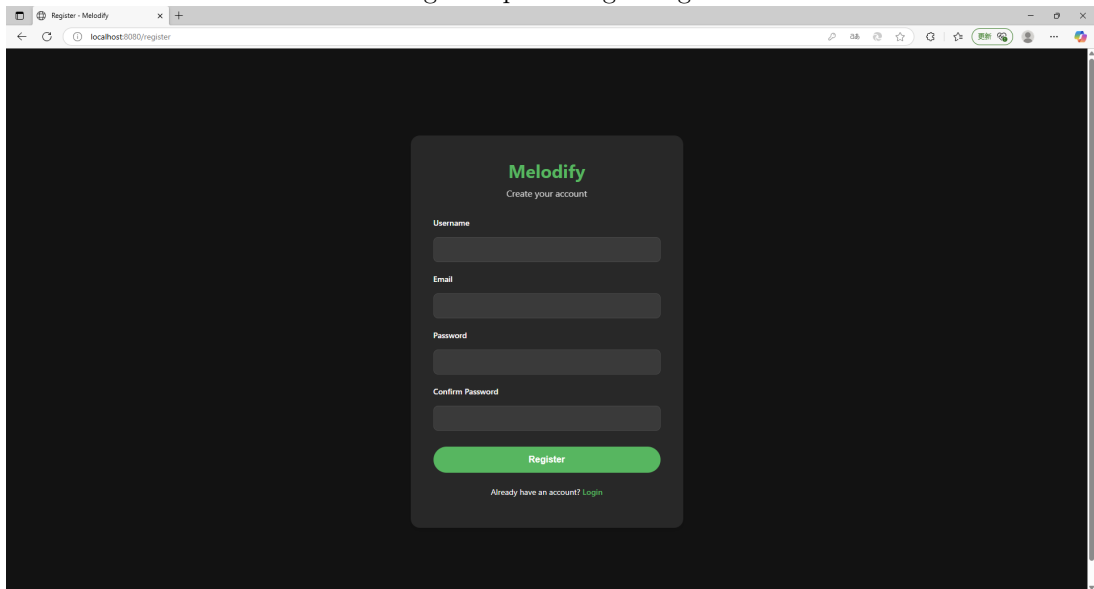


Figure Apd.3: Register Page

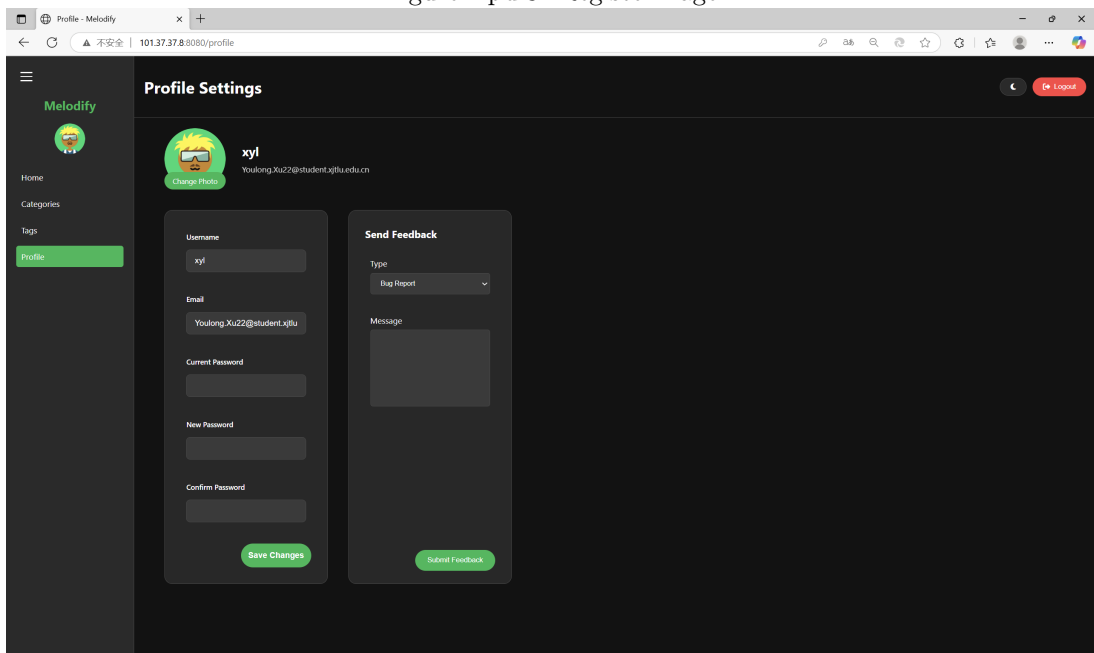


Figure Apd.4: User Profile Page

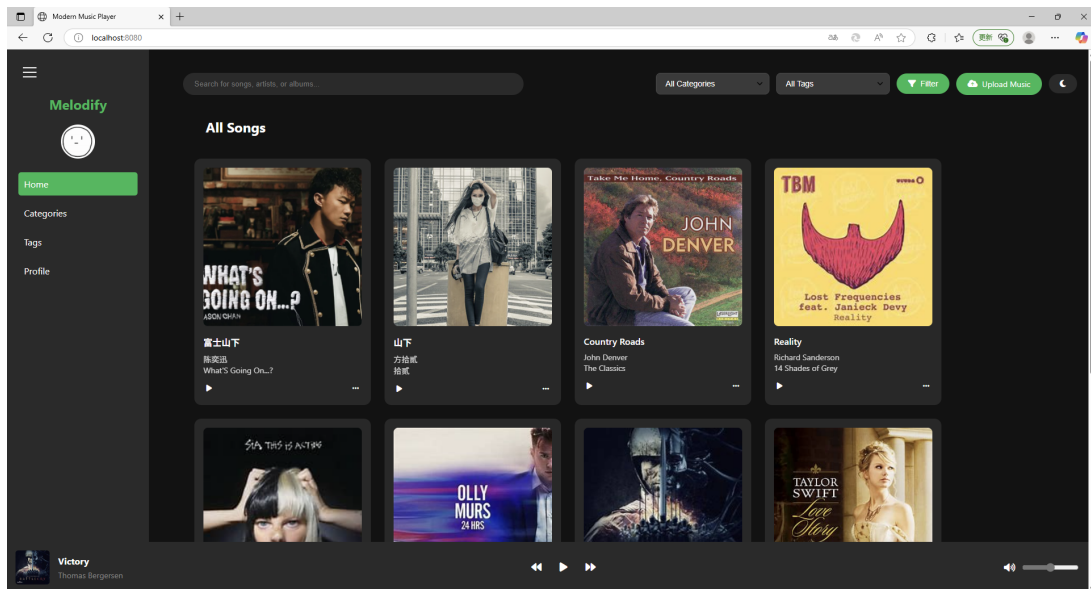


Figure Apd.5: Index Page

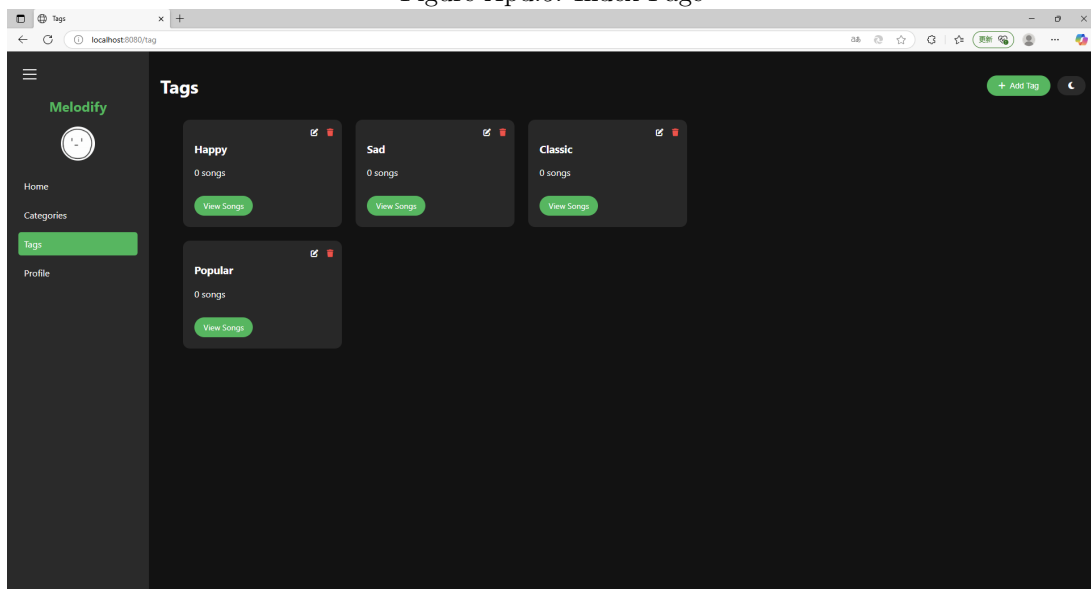


Figure Apd.6: Tag Page

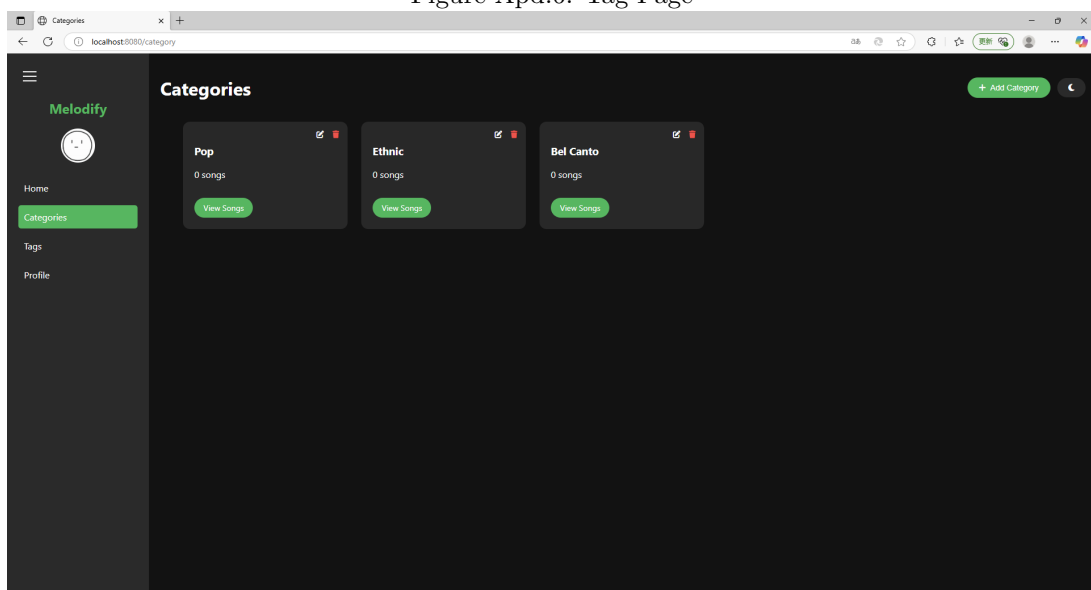


Figure Apd.7: Category Page

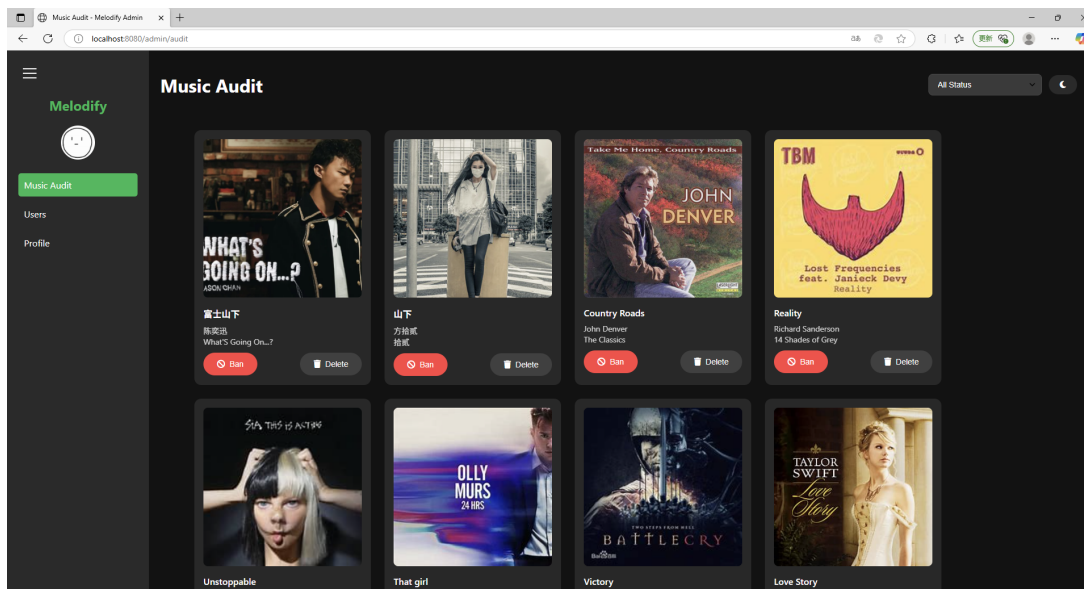


Figure Apd.8: Admin-audit Page

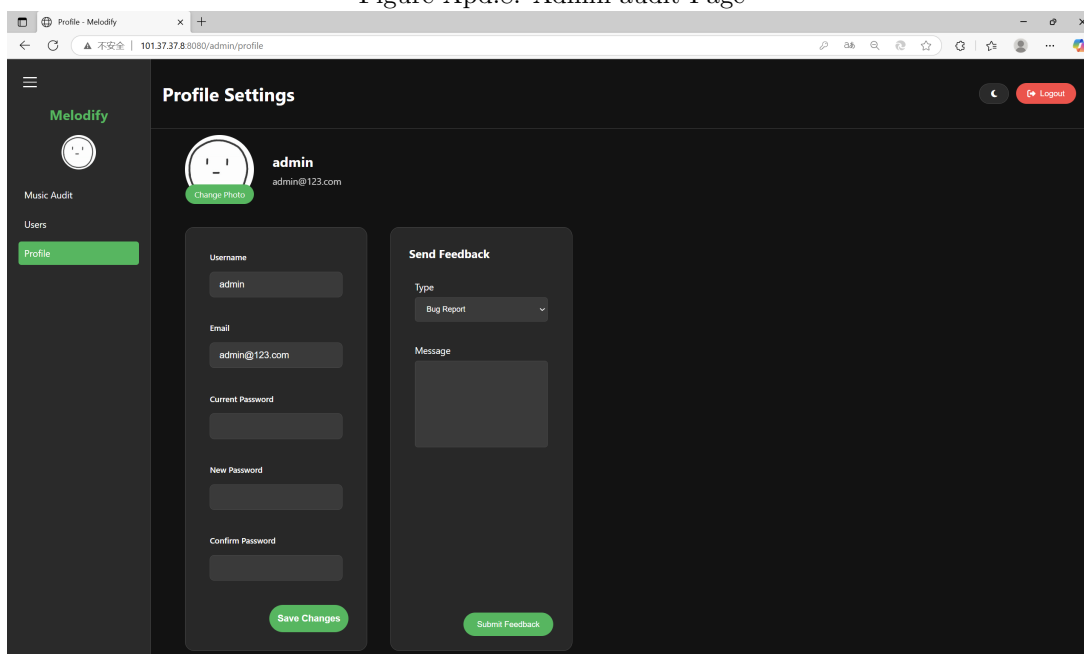


Figure Apd.9: Admin-profile Page

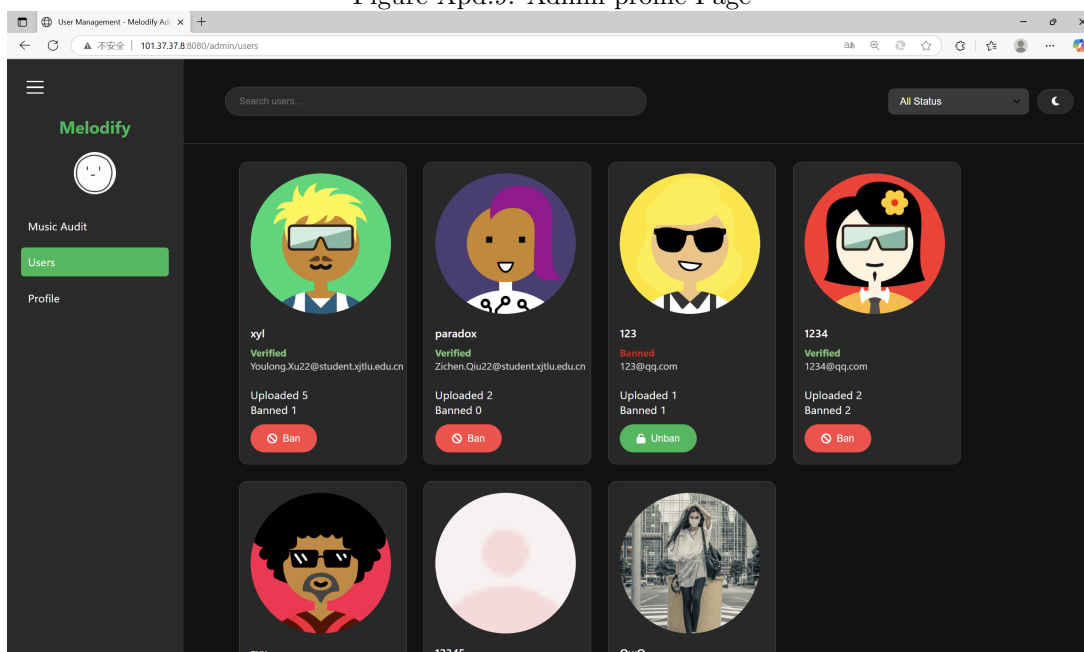


Figure Apd.10: Admin-users Page

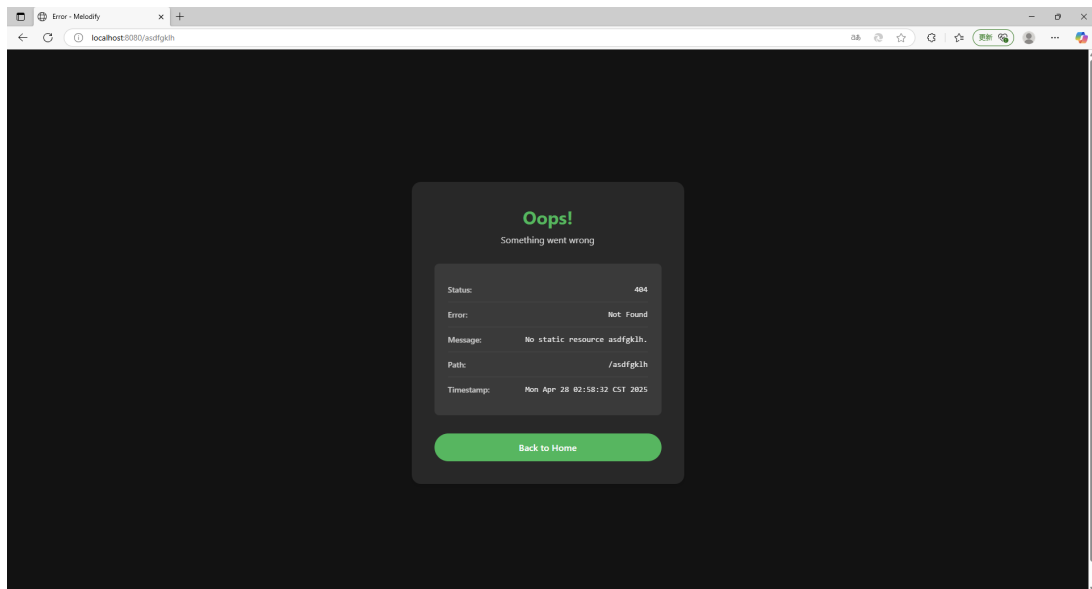


Figure Apd.11: Error Page

Appendix D: Configuration Files

```
1
2 services:
3   nginx:
4     image: nginx:latest
5     container_name: nginx
6     ports:
7       - 80:80
8       - 443:443
9     volumes:
10      # WorkDir:/
11      - /cpt202/nginx/cert:/etc/nginx/cert
12      - /cpt202/nginx/conf/nginx.conf:/etc/nginx/nginx.conf
13      - /cpt202/nginx/html:/usr/share/nginx/html
14      - /cpt202/nginx/log:/var/log/nginx
15      # - /etc/localtime:/etc/localtime:ro
16     privileged: true
17     networks:
18       - cpt202_net
19
20   cpt202-server:
21     image: server
22     build:
23       context: .
24     container_name: server
25     environment:
26       - TZ=Asia/Shanghai
27       - SPRING_PROFILES_ACTIVE=prod
28     volumes:
29      # WorkDir:/cpt202/data
30      - /cpt202/data/logs:/cpt202/data/logs:rw
31      - /cpt202/data/music:/cpt202/data/music:rw
32      - /cpt202/data/img:/cpt202/data/img:rw
33      - /cpt202/run.sh:/cpt202/run.sh
34     command: "./run.sh start"
35     ports:
36       - 8080:8080
37     privileged: true
38     restart: always
39     networks:
40       cpt202_net:
41         ipv4_address: 172.30.0.4
42
43   mysql:
44     image: mysql:8.0.18
45     container_name: mysql
46     ports:
47       - 3306:3306
48     environment:
49       - MYSQL_ROOT_PASSWORD=wEJD}VEy2*
50       - MYSQL_DATABASE=cpt202
51     command: --lower_case_table_names=1
52     volumes:
53       - /cpt202/cpt202.sql:/docker-entrypoint-initdb.d/init.sql
54       - /cpt202/mysql/log:/var/log/mysql
55     restart: always
56     privileged: true
57     networks:
58       cpt202_net:
59         ipv4_address: 172.30.0.5
60
61   redis:
62     image: redis:6.2.12
63     container_name: redis
64     ports:
65       - 6379:6379
```

```

66     environment:
67         TZ: Asia/Shanghai
68         LANG: en_US.UTF-8
69     volumes:
70         # WorkDir:/data
71         - /cpt202/redis/conf/redis.conf:/etc/redis/redis.conf:rw
72         - /cpt202/redis/data:/data
73     command: /bin/sh -c "chmod 777 /data && redis-server --appendonly yes"
74     privileged: true
75     restart: always
76     networks:
77         cpt202_net:
78             ipv4_address: 172.30.0.6
79
80 networks:
81     cpt202_net:
82         driver: bridge
83         ipam:
84             config:
85                 - subnet: 172.30.0.0/16

```

Listing 1: docker-compose.yml

```

1  FROM adoptopenjdk/openjdk17:alpine
2
3  MAINTAINER Xu
4
5  RUN mkdir -p /cpt202/data/logs
6  RUN mkdir -p /cpt202/data/img
7  RUN mkdir -p /cpt202/data/music
8
9  WORKDIR /cpt202/data
10
11 EXPOSE 8080
12
13 ADD ./cpt202.jar ./cpt202.jar
14 ADD ./run.sh ./run.sh
15
16 RUN ["chmod", "+x", "./run.sh"]

```

Listing 2: Dockerfile

Appendix E: Testing Code and Result

This appendix presents the key unit test procedures for the 'UserController' class, which manages user registration, login, and verification. The tests ensure that the controller behaves as expected across various scenarios.

```
1 public class UserControllerTest {
2
3     @InjectMocks
4     private UserController userController;
5
6     @Mock
7     private UserRepo userRepo;
8     @Mock
9     private UserService userService;
10
11     private MockMvc mockMvc;
12
13     @BeforeEach
14     public void setUp() {
15         mockMvc = MockMvcBuilders.standaloneSetup(userController).build();
16     }
17     // Test: Register page load
18     @Test
19     public void testGetRegisterPage() throws Exception {
20         mockMvc.perform(MockMvcRequestBuilders.get("/register"))
21             .andExpect(MockMvcResultMatchers.status().isOk())
22             .andExpect(MockMvcResultMatchers.view().name("registerPage"));
23     }
24     // Test: Successful registration
25     @Test
26     public void testPostRegisterPage_Success() throws Exception {
27         User user = new User();
28         user.setEmail("test@example.com");
29         user.setPassword("Password123!");
30         when(userRepo.findByEmail(user.getEmail())).thenReturn(null);
31         doNothing().when(userService).register(any(User.class), any(MultipartFile.class));
32
33         MockMultipartFile mockFile = new MockMultipartFile("avatarFile", "test-avatar.jpg",
34             "image/jpeg", "test".getBytes());
35
36         mockMvc.perform(MockMvcRequestBuilders.multipart("/register")
37             .file(mockFile)
38             .param("email", user.getEmail())
39             .param("password", user.getPassword()))
40             .andExpect(MockMvcResultMatchers.status().isOk())
41             .andExpect(MockMvcResultMatchers.view().name("registerPage"))
42             .andExpect(MockMvcResultMatchers.model().attribute("message", "Registration
43                 successful."));
44     }
45     // Test: Email already registered
46     @Test
47     public void testPostRegisterPage_EmailAlreadyRegistered() throws Exception {
48         User user = new User();
49         user.setEmail("test@example.com");
50         when(userRepo.findByEmail(user.getEmail())).thenReturn(new User());
51
52         mockMvc.perform(MockMvcRequestBuilders.post("/register")
53             .param("email", user.getEmail())
54             .param("password", "Password123!"))
55             .andExpect(MockMvcResultMatchers.status().isOk())
56             .andExpect(MockMvcResultMatchers.model().attribute("error", "Email already
57                 registered"));
```

Listing 3: Unit Test for UserController

Below are the results for the unit tests performed on different Controllers and Services:

✓ UserControllerTest (com.cpt202.group7.controller) 1 sec 346 ms	
✓ testPostRegisterPage_Success()	1 sec 68 ms
✓ testPostLoginPage_Success()	232 ms
✓ testVerifyUser_Failure()	10 ms
✓ testPostRegisterPage_EmailAlreadyRegistered()	7 ms
✓ testGetRegisterPage()	7 ms
✓ testGetAvatar()	15 ms
✓ testVerifyUser_Success()	7 ms

Figure Apd.12: User Controller Test

✓ AdminControllerTest (com.cpt202.group7.controller) 1 sec 215 ms	
✓ lockUser_AdminLockNormalUser_UpdatesStatus()	1 sec 151 ms
✓ lockUser_AdminLockAdmin_ReturnsError()	13 ms
✓ viewFile_ValidRequest_ReturnsFile()	16 ms
✓ getAdminPage_AdminUser_ReturnsAuditPage()	9 ms
✓ users_AdminUser_ReturnsUsersPage()	9 ms
✓ getAdminPage_NonAdminUser_RedirectToLogin()	8 ms
✓ updateProfile_ValidUpdate_UpdatesUser()	9 ms

Figure Apd.13: Admin Controller Test

✓ BaseControllerTest (com.cpt202.group7.controller) 1 sec 139 ms	
✓ testErrorPage()	1 sec 131 ms
✓ testDemoPage()	8 ms

Figure Apd.14: Base Controller Test

✓ CategoryControllerTest (com.cpt202.group7.controller) 1 sec 118 ms	
✓ testAddCategory()	1 sec 91 ms
✓ testGetAllCategories()	10 ms
✓ testEditCategory()	10 ms
✓ testDeleteCategory()	7 ms

Figure Apd.15: Category Controller Test

✓ FeedbackControllerTest (com.cpt202.group7.controller) 957 ms	
✓ testGetFeedbackPage_UserRoleIsNot0_ShouldReturnFeedbackPage()	929 ms
✓ testPostFeedback_UserRoleIs0_ShouldRedirectToHomePage()	16 ms
✓ testGetFeedbackPage_UserRoleIs0_ShouldRedirectToHomePage()	7 ms
✓ testPostFeedback_UserRoleIsNot0_ShouldReturnFeedbackPage()	5 ms

Figure Apd.16: Feedback Controller Test

✓ MusicPlayControllerTest (com.cpt202.group7.controller) 817 ms	
✓ testPlay_WithPartialParameters()	807 ms
✓ testPlay_WithMissingParameters()	5 ms
✓ testPlay_WithValidParameters()	5 ms

Figure Apd.17: Music Play Controller Test

✓ TagControllerTest (com.cpt202.group7.controller) 684 ms	
✓ testAddCategory()	672 ms
✓ testGetAllCategories()	6 ms
✓ testEditCategory()	3 ms
✓ testDeleteCategory()	3 ms

Figure Apd.18: Tag Controller Test

✓ UserProfileControllerTest (com.cpt202.group7.controller) 1 sec 127 ms	
✓ testUpdateProfile_Success()	1 sec 101 ms
✓ testViewProfile_Success()	11 ms
✓ testDownloadFile_Success()	15 ms

Figure Apd.19: User Profile Controller Test

✓ MusicResourceControllerTest (com.cpt202.group7.controller) 728 ms	
✓ testAddMusicResource_NewMusic()	724 ms
✓ testEditMusicResource()	4 ms

Figure Apd.20: Music Resource Controller Test

✓ UploadControllerTest (com.cpt202.group7.controller) 1 sec 117 ms	
✓ testUploadMusic(Path)	1 sec 117 ms

Figure Apd.21: Upload Controller Test

✓ SearchControllerTest (com.cpt202.group7.controller) 1 sec 129 ms	
✓ testFilterCategories_CategoryAndTags()	1 sec 102 ms
✓ testFilterCategories_TagOnly()	8 ms
✓ testGetSearchResult()	7 ms
✓ testFilterCategories_CategoryOnly()	6 ms
✓ testFilterCategories_AllNull()	6 ms

Figure Apd.22: Search Controller Test

✓ UserServiceImplTest (com.cpt202.group7.service) 1 sec 198 ms	
✓ testVerifyUser()	691 ms
✓ testSendRegistrationConfirmationEmail()	23 ms
✓ testRegister()	476 ms
✓ testUploadAvatarFile()	4 ms
✓ testUploadAvatarFile_FileSaveFailure()	4 ms

Figure Apd.23: User Service Test

✓ EmailServiceImplTest (com.cpt202.group7.service.impl) 891 ms	
✓ testSendMail()	891 ms

Figure Apd.24: Email Service Test

✓ SecureTokenServiceImplTest (com.cpt202.group7.service.impl) 638 ms	
✓ testFindByTokenAndUserId_NoMatchingToken()	626 ms
✓ testSaveSecureToken()	2 ms
✓ testCreateToken()	3 ms
✓ testRemoveToken()	1 ms
✓ testFindByTokenAndUserId_MatchingToken()	3 ms
✓ testFindByToken()	2 ms
✓ testFindByTokenAndUserId_NoTokens()	1 ms

Figure Apd.25: Secure Token Service Test

Appendix F: Sprint Backlogs

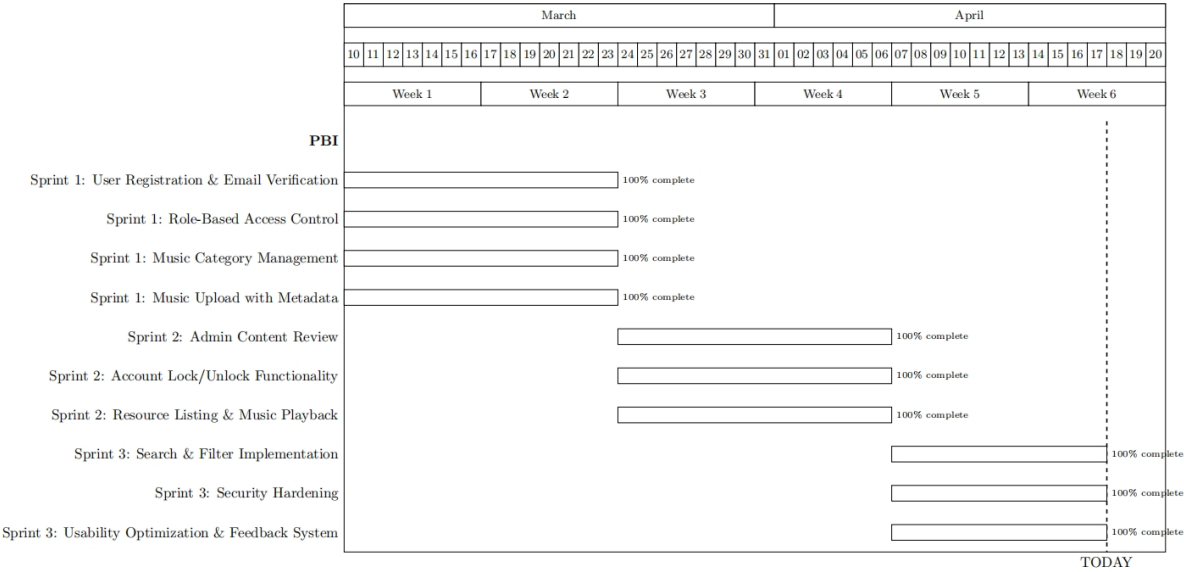
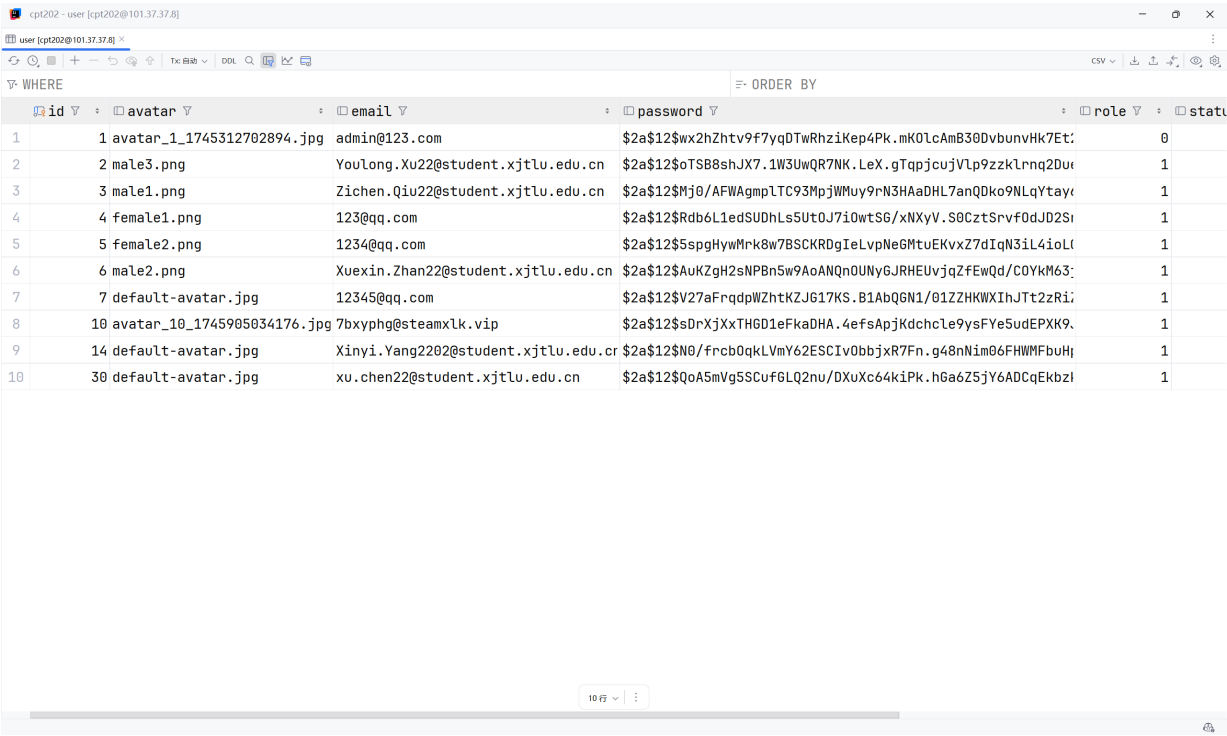


Figure Apd.26: Sprint Backlogs(Made in 2025/4/17

Appendix G: Secure encryption



The screenshot shows a web browser window with a table of user data. The table has columns for id, avatar, email, password, role, and status. The passwords are displayed as long, alphanumeric strings, indicating they have been encrypted. The browser's address bar shows the URL 'user [cpt202@101.37.37.8]'.

id	avatar	email	password	role	status
1	avatar_1_1745312702894.jpg	admin@123.com	\$2a\$12\$wx2hZhtv9f7yqDTwRhziKep4Pk.mK0lcAmB30DvbunvHk7ET:	0	
2	male3.png	Youlong.Xu22@student.xjtlu.edu.cn	\$2a\$12\$oTSB8shJX7.1W3UwQR7NK.LeX.gTqpjcuVlP9zzkLrnq2D0u:	1	
3	male1.png	Zichen.Qiu22@student.xjtlu.edu.cn	\$2a\$12\$Mj0/AFWAgmpLTC93MpjWMuy9rN3HAaDHL7anQDKo9NLqYtay:	1	
4	female1.png	123@qq.com	\$2a\$12\$Rdb6L1edSUDhLs5Ut0J7i0wtSG/xNXyV.S0CztSrvf0dJD2Si:	1	
5	female2.png	1234@qq.com	\$2a\$12\$5spgHywMrk8w7BCKRDgIeLvpNeGmtuEKvxZ7dIqN3iL4ioLl:	1	
6	male2.png	Xuexin.Zhan22@student.xjtlu.edu.cn	\$2a\$12\$AuKZgH2sNPBn5w9AoANQn0UNy6JRHEUvjQZfEwQd/COYkM63:	1	
7	default-avatar.jpg	12345@qq.com	\$2a\$12\$V27aFrqdpWZhtKZJ617KS.B1AbQ6N1/01ZZHKWIXihJTt2zRi:	1	
8	avatar_10_1745905034176.jpg	7bxyphg@steamXk.vip	\$2a\$12\$DrXjXxTHG01eFkaDHA.4efsApjKdchcLe9ysFye5udePXX9:	1	
9	default-avatar.jpg	Xinyi.Yang2202@student.xjtlu.edu.cn	\$2a\$12\$N0/frcb0qkLVmY62ESCiv0bbjxR7Fn.g48nNim06FHWMFbuHj:	1	
10	default-avatar.jpg	xu.chen22@student.xjtlu.edu.cn	\$2a\$12\$QoA5mVg5SCuf6LQ2nu/DXuXc64kiPk.hGa6Z5jY6ADCqEkbzI:	1	

Figure Apd.27: Passwords secure encryption

Appendix H: Member Contributions Form

The contribution of each individual is identical, each accounting for 12.5%.